

1. Array vs Collection

Feature	Array	Collection
Size	Fixed size (static)	Dynamic size (grows/shrinks automatically)
Data Type	Can hold both primitive and object types	Can only hold objects (primitives need wrapper classes)
Memory	Contiguous memory allocation	Uses data structures with various memory patterns
Performance	Faster for basic operations	Slightly slower due to dynamic resizing
Type Safety	Type-safe at compile time	Generic collections are type-safe
Methods	Limited (length property, clone)	Rich API (add, remove, contains, etc.)
Dimension	Supports multi-dimensional arrays	Primarily single dimension (can nest collections)

java

```
// Array Example
int[] numbers = new int[5];           // Fixed size
numbers[0] = 10;                      // Can store primitives
int length = numbers.length;          // length property

// Collection Example
ArrayList<Integer> list = new ArrayList<>(); // Dynamic size
list.add(10);                          // Autoboxing primitive to object
int size = list.size();                 // size() method
```

2. ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Internal Structure	Dynamic array	Doubly linked list
Access Time	O(1) - Fast random access	O(n) - Slow random access
Insertion/Deletion	O(n) - Slow (requires shifting)	O(1) - Fast (just change references)
Memory Overhead	Less (only stores data)	More (stores previous/next pointers)
Best For	Retrieval operations	Insertion/deletion operations
Implements	List interface only	List and Deque interfaces
Capacity	Has capacity concept	No capacity concept

java

```
// ArrayList - Fast access, slow modification
ArrayList<String> arrayList = new ArrayList<>();
arrayList.add("A");    // Adds at end
arrayList.get(0);      // Fast random access

// LinkedList - Fast modification, slow access
LinkedList<String> linkedList = new LinkedList<>();
linkedList.add("A");    // Adds at end
linkedList.addFirst("B"); // Add at beginning - O(1)
linkedList.get(0);      // Slow - traverses from start/end
```

3. List vs Set

Feature	List	Set
---------	------	-----

Duplicates	Allows duplicates	No duplicates allowed
Order	Maintains insertion order	No guaranteed order (except LinkedHashSet, TreeSet)
Null Elements	Multiple nulls allowed	At most one null (except TreeSet)
Index-Based Access	Yes (get(index))	No direct index access
Common Implementations	ArrayList, LinkedList, Vector	HashSet, LinkedHashSet, TreeSet
Sorting	Can be sorted using Collections.sort()	TreeSet is sorted, others not
Positional Access	Supported	Not supported

java

```
// List - Allows duplicates, maintains order
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("A");          // Duplicate allowed
System.out.println(list); // [A, B, A]

// Set - No duplicates, no order guarantee
Set<String> set = new HashSet<>();
set.add("A");
set.add("B");
set.add("A");           // Duplicate ignored
System.out.println(set); // [A, B] (order not guaranteed)
```

4. Collection vs Collections

Feature	Collection	Collections
Type	Interface	Utility class
Purpose	Root interface of collection hierarchy	Provides static utility methods
Package	java.util.Collection	java.util.Collections
Instantiation	Can be implemented/instantiated	Cannot be instantiated (static methods)
Common Methods	add(), remove(), size(), iterator()	sort(), reverse(), shuffle(), binarySearch()
Part Of	Collection Framework core	Helper class for operations

java

```
// Collection - Interface
Collection<String> collection = new ArrayList<>();
collection.add("A");    // Interface method
collection.remove("A");
collection.size();

// Collections - Utility class with static methods
List<String> list = Arrays.asList("C", "A", "B");
Collections.sort(list);    // Static utility method
Collections.reverse(list);
Collections.shuffle(list);
Collections.binarySearch(list, "B");
```

5. Comparable vs Comparator

Feature	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(T o)	compare(T o1, T o2)
Sorting	Single sorting sequence	Multiple sorting sequences
Modification	Modifies the original class	External class - doesn't modify original
Implementation	Implemented by the class being compared	Separate class or lambda expression
Default Sort	Natural/default ordering	Custom ordering
Syntax	Collections.sort(list)	Collections.sort(list, comparator)

java

```
// Comparable - Natural ordering (implemented in the class)
public class Student implements Comparable<Student> {
    int id;
    String name;

    @Override
    public int compareTo(Student s) {
        return this.id - s.id; // Sort by id
    }
}

// Usage
Collections.sort(students); // Uses compareTo

// Comparator - Custom ordering
```

```

class NameComparator implements Comparator<Student> {
    @Override
    public int compare(Student s1, Student s2) {
        return s1.name.compareTo(s2.name); // Sort by name
    }
}

class IdComparator implements Comparator<Student> {
    @Override
    public int compare(Student s1, Student s2) {
        return s1.id - s2.id; // Alternative id sorting
    }
}

// Usage
Collections.sort(students, new NameComparator()); // Sort by name
Collections.sort(students, new IdComparator());    // Sort by id

// Lambda expression (modern approach)
Collections.sort(students, (s1, s2) -> s1.name.compareTo(s2.name));

```

Quick Reference Table

Feature	Array	Collection	ArrayList	LinkedList	List	Set	Collection	Collections	Comparable	Comparator
Type	Data structure	Interface	Class	Class	Interface	Interface	Interface	Class	Interface	Interface
Duplicates	Yes	Varies	Yes	Yes	Yes	No	Varies	N/A	N/A	N/A

Order	Yes	Varies	Yes	Yes	Yes	No	Varies	N/A	N/A	N/A
Null-Safe	Yes	Varies	Yes	Yes	Yes	Limited	Varies	N/A	No	Yes

Practical Example: Combining Concepts

java

```
public class ComparisonDemo {
    public static void main(String[] args) {
        // Array vs Collection
        int[] array = {3, 1, 2};
        List<Integer> collection = new ArrayList<>(Arrays.asList(3, 1, 2));

        // ArrayList vs LinkedList
        List<String> arrayList = new ArrayList<>();
        List<String> linkedList = new LinkedList<>();

        // List vs Set
        List<String> list = new ArrayList<>();
        Set<String> set = new HashSet<>();

        // Collection vs Collections
        Collection<String> coll = new ArrayList<>();
        Collections.sort(new ArrayList<>()); // Collections utility

        // Comparable vs Comparator
    }
}
```

```
List<String> names = Arrays.asList("Charlie", "Alice", "Bob");  
Collections.sort(names); // Natural order (Comparable)  
Collections.sort(names, (a, b) -> b.compareTo(a)); // Reverse order (Comparator)  
}  
}
```